

Independent Audit of the Nopert Certificate for the Rhombicosidodecahedron

David Ryan

Version 0.1.0, 2026-10-08

DOI: 10.5281/zenodo.23232399

Abstract

Bence Hervay announced a computer-assisted proof that the rhombicosidodecahedron has no Rupert passage. This report gives an independent mathematical and computational audit of that claim. It uses *Nopert* for a solid without Rupert's property.

The audit examined the mathematical reduction before running the supplied programs. It then reproduced the Rust checker and Python audit from commit `802a3ded09535c4a99cef1371ce0d0277c433fa8`. Both accepted all 192,696 certificate records. A clean Rust search regenerated the 18,195,036-byte certificate exactly. Its SHA-256 hash matched the supplied certificate.

The review concentrated on the zoom lemma and the reduction from all possible passages to a finite local cover. No mathematical gap was found. The repository's exhaustive second Rust model also rebuilt and accepted all 11,183 zoom cells.

Outcome: confirmed subject to stated assumptions.

The strongest further assurance would be an externally authored clean-room checker for the 38 zoom covers. The supplied Python checker treats those cover proofs as inputs. Hervay's second Rust model checks them independently at the proof-structure level, while sharing the repository's arithmetic layer.

1. Scope and disclosure

This report audits Hervay's stated theorem, its mathematical reduction, its exact certificate, and the supplied checking software. It does not assess the result's novelty or every historical statement in the article.

This is an independent technical audit. It is not peer review or a formal proof. Independent means that the audit was conducted separately from the author and reasoned from the mathematical argument before trusting program outputs.

David Ryan directed the audit. OpenAI Codex using GPT-5.6 Sol at Extra High reasoning performed the mathematical review, source inspection, computational reproduction, adversarial analysis, and first report draft on 2026-10-08.

Every quantitative claim below comes from an executable check, retained output, or named source file. The repository accompanying this report records the commands, hashes, environment, and limitations.

2. Audited material

- Bence Hervay, *Non-Rupertness of the Rhombicosidodecahedron*, 58 pages.
- Repository: bence-hervay/nopert-rid.
- Audited commit: 802a3ded09535c4a99cef1371ce0d0277c433fa8.
- Projection source: Steininger and Yurkevich, *An algorithmic approach to Rupert's problem*.

The checkout had no tag at its head. The full commit identifies the audited version.

Input	SHA-256
paper/article.pdf	e52022737fad1f5add102b8a8b357788a3c5086111f028255a6e7a26d6bd71ff
proof/results/full/search.cert	df1f8dbab68b561f22fa9daeca48153d71bde7b6fba3345d9f6b7b8077df4b5f
proof/Cargo.lock	2f40f55632dcbccd722fcd2681153f1f5472984680680d3346dbcee4dbb1bd24
audit/uv.lock	2ea8eda0baf9f33ef03a0a8c946860b2f3def1eaffe154ba27d22f9b4f4bc44b

3. Audit method

The audit used four layers.

First, it read the article's argument and traced each reduction into the code. This review covered projected containment, centring, symmetry, branch and bound, exact arithmetic, local exclusions, and certificate completeness.

Second, it built and ran the supplied Rust checker and Python audit from a clean checkout. The environments used locked Rust and Python dependencies.

Third, it regenerated the complete certificate with the Rust search. A byte comparison tested whether the clean search reproduced the supplied object.

Fourth, it attacked the proof's choke points. The audit inspected cover completeness, divisibility factors, no-fit sets, support conditions, hand-over maps, and domain extensions. It also ran the repository's exhaustive second cover model and the supplied mutation controls.

4. Mathematical audit

4.1 Projection and centring

The projection criterion reduces a straight passage to strict containment of two planar shadows. Central symmetry then removes planar translation. The midpoint proof of the centring lemma works for ordinary and strict containment.

The margin function correctly expresses scaled radial containment. It varies continuously because each shadow has finitely many projected vertices. Aligned configurations have margin one. Excluding every strict fit therefore proves the Nopert result.

The projection and centring reductions agree with Propositions 1 and 2 of Steininger and Yurkevich.

4.2 Symmetry-reduced domain

The viewing-line reduction uses all 120 orthogonal body symmetries. Their retained half-spaces give the stated triangular viewing cone.

The relative rotation reduction compares the rotation with its images under the full icosahedral rotation group and the central half-turn. The necessary comparisons ensure that every physical configuration has a representative in the closed domain D .

The proof only needs D to contain one representative of every configuration. Uniqueness and minimality are unnecessary.

4.3 Global component

A projected plug vertex beyond a valid support line excludes containment. The support normal remains perpendicular to the changing view. Exact Bernstein signs certify separation throughout each closed search box.

The final certificate does not depend on the Rust checker's support shortcut. The Python replay validated the full sixty-member maximum form for all 157,458 Global records. The shortcut also held for 908 records.

4.4 Zoom lemma

The zoom lemma is sound when five obligations hold:

1. Each divided coordinate is non-negative on the cell.
2. Its zero set maps into a proved no-fit set.
3. The pulled-back witness is exactly divisible by the stated monomial.
4. Every support condition holds at each required corner view.
5. Every Bernstein coefficient of the quotient has the required strict sign.

The corner step is valid because each support condition is multi-affine in the zoom coordinates. Its minimum on a box occurs at a corner. Exact division removes only non-negative distance factors. A zero factor places the configuration in the applicable no-fit set.

The implementation checks each obligation. It rejects negative scale ranges, wrong factors, failed exact division, non-positive views, invalid supports, and non-strict quotient signs.

4.5 No-fit sets and local covers

The aligned set contains no fit because both shadows coincide. The two arc planes contain no fit because both shadows have the same support value in the screen direction e_2 .

Tube coverage normalises each non-zero offset on a face of its sign box. Point coverage divides cases by the ratio of offset size to base size. Together they cover the stated product box, including the shared boundary.

The crossing hand-over is complete. Each delegated first-arc cell maps to the same physical configuration in a sheared second-arc zoom. The exact window test keeps the image inside that zoom's root.

The pentagon cover extends beyond one face of its proved region. The code checks that domain inequality zero is a positive multiple of the distance beyond that face. The extension therefore contains only points outside D.

4.6 Reduction to a finite certificate

The binary certificate leaves cover the whole five-dimensional root box. The proof uses two checked facts for every leaf:

- The leaf belongs to a sound elimination component.
- The leaves form a complete, prefix-free binary cover of the root.

These facts close the risk of an unlisted passage between local models. Any passage would lie in a certificate leaf. That leaf's exact component proof would give a contradiction.

5. Reproduction results

Check	Result
Certificate framing	192,696 records; 18,195,036 bytes; final newline present
Component counts	12,650 Domain; 157,458 Global; 21,792 Local; 796 Exotic
Rust checker	complete:true; zero frontier; 192,696 records accepted
Rust checker time	49.26 seconds
Python record replay	192,696 accepted; zero refused; 116 seconds
Python structure check	Prefix-free; complete; Kraft sum one; 20,000 probes covered
Clean Rust search	385,391 boxes evaluated; 192,696 records; 89.764 seconds
Search comparison	Byte-identical to the supplied certificate
Ordinary Rust tests	393 proof and integration tests passed
Exhaustive second cover model	Four partitions passed; 11,183 zoom cells accepted

The regenerated certificate had SHA-256 `df1f8dbab68b561f22fa9daeca48153d71bde7b6fba3345d9f6b7b8077df4b5f`. This matched the supplied certificate.

The Python replay output had SHA-256 `b46d5c27f030c893c67bf9b467e00fd700acc5fe684478da22db3e038790a07`.

The first audit ran on Darwin 25.6.0 arm64. It used Rust and Cargo 1.99.0, Python 3.14.6, NumPy 2.5.3, and python-flint 0.9.0. The temporary checkout, toolchains, build outputs, and caches used less than 1 GB.

6. Adversarial checks

The supplied Python controls made the following attacks:

- 4,800 Global witnesses on boxes containing an aligned rotation produced zero false acceptances.
- 400 Global witnesses on boxes containing a symmetry rotation produced zero false acceptances.
- 10,950 Domain claims on boxes containing a point of D produced zero false acceptances.
- 11,400 cover-containment probes produced zero disagreements with corner sampling.
- Foreign, reversed, and enlarged Global witnesses failed unless the witness remained valid.

The Rust tests mutate records, witnesses, checksums, cover cells, tilings, factors, zoom maps, window delegations, and inclusion boxes. The checker refused corrupt inputs or independently confirmed the surviving claim.

The exhaustive ignored test reconstructs each cover from its definitions. It rebuilds zoom families, witness polynomials, monomial division, Bernstein coefficients, cell trees, the hand-over, and the pentagon extension. All four partitions passed in 515.21 seconds.

7. Qualifications and assumptions

The default Rust test command reported two failures on the audit Mac. One test uses Linux `prlimit`, which macOS lacks. One scheduler performance test crossed its timing threshold in the sandbox. Both concern failure handling or worker utilisation. Neither enters the mathematical proof checker.

Every other ordinary unit and integration test passed after those exclusions. The proof-relevant exhaustive test also passed.

The Python checker independently verifies certificate records, framing, and coverage. It treats the zoom-cover cell proofs as checked inputs. The repository's exhaustive second Rust model checks those cells through a separate proof implementation. It still shares the repository and low-level arithmetic layer.

The verdict assumes:

- The audited commit is the version attached to the announced claim.
- Rust's `num` crates correctly implement arbitrary-precision arithmetic.
- Python `FLINT` correctly implements integer and rational arithmetic.
- The operating system and compiler executed the checked source faithfully.

Cross-language agreement reduces arithmetic risk. A formal kernel or an externally authored cover checker would provide stronger assurance.

8. Conclusion

The mathematical reduction survived adversarial review. Both supplied checkers accepted every certificate record. The clean search reproduced the certificate exactly. The exhaustive second model accepted every zoom cell.

The audit therefore confirms Hervay's proof that the rhombicosidodecahedron is Nopert, subject to the stated assumptions.

The next useful contribution is a clean-room implementation of the 38 zoom covers. It should use its own coordinate maps, polynomial division, and cover logic. That implementation would reduce the remaining shared-code risk and provide a reusable checker architecture for future Nopert results.

References

1. Bence Hervay. *Non-Rupertness of the Rhombicosidodecahedron*. Source and proof repository, audited commit 802a3ded09535c4a99cef1371ce0d0277c433fa8, 2026.
2. Jakob Steininger and Sergey Yurkevich. *An algorithmic approach to Rupert's problem*. arXiv:2112.13754, 2021. Revised publication metadata appears on the arXiv record.